Compiler Design Lab

Date: 27-2-2026 **Name:** S Vishwajith
Assignment No.: 8 **Register No.:** 23BCE1145

Aim:

To implement a CLR parser using C++.

Code:

```
#include <iostream>

#include <vector>

#include <string>

#include <map>

#include <set>

#include <stack>

#include <algorithm>

#include <iomanip>

using namespace std;


vector<pair<string, vector<string>>> G = {

  {"S'", {"S"}},

  {"S", {"C", "C"}},

  {"C", {"c", "C"}},

  {"C", {"d"}},

};


const string INPUT = "ccdd";

set<string> NT, TM;

string AUG;
```

```
bool isNT(const string &s) { return NT.count(s) > 0; }

void deriveSymbols()
{
   AUG = G[0].first;
   for (auto &[l, r] : G)
      NT.insert(l);
   for (auto &[l, r] : G)
      for (auto &s : r)
         if (!NT.count(s))
            TM.insert(s);
   TM.insert("$");
}

map<string, set<string>> FIRST;

set<string> firstSeq(const vector<string> &v, int from = 0)
{
   set<string> res;
   if (from >= (int)v.size())
   {
      res.insert("eps");
      return res;
   }
   for (int i = from; i < (int)v.size(); i++)
   {
      const string &s = v[i];
```

```
        set<string> fi = TM.count(s) ? set<string>{s} : FIRST[s];
        bool hasEps = fi.count("eps");
        for (auto &x : fi)
            if (x != "eps")
                res.insert(x);
        if (!hasEps)
            return res;
    }
    res.insert("eps");
    return res;
}

void computeFirst()
{
    for (auto &n : NT)
        FIRST[n] = {};
    bool changed = true;
    while (changed)
    {
        changed = false;
        for (auto &[l, r] : G)
            for (auto &x : firstSeq(r))
                if (!FIRST[l].count(x))
                {
                    FIRST[l].insert(x);
                    changed = true;
                }
    }
```

```
}

struct Item
{
   int p, d;
   string la;
   bool operator<(const Item &o) const { return tie(p, d, la) < tie(o.p, o.d, o.la); }
   bool operator==(const Item &o) const { return p == o.p && d == o.d && la == o.la; }
};
using ISet = set<Item>;

string itemStr(const Item &it)
{
   auto &[lhs, rhs] = G[it.p];
   string s = lhs + " ->";
   for (int i = 0; i <= (int)rhs.size(); i++)
   {
      if (i == it.d)
         s += " .";
      if (i < (int)rhs.size())
         s += " " + rhs[i];
   }
   s += "  [" + it.la + "]";
   return s;
}

ISet closure1(ISet s)
{
```

```
vector<Item> wl(s.begin(), s.end());
while (!wl.empty())
{
    Item it = wl.back();
    wl.pop_back();
    auto &rhs = G[it.p].second;
    if (it.d >= (int)rhs.size())
        continue;
    const string &B = rhs[it.d];
    if (!isNT(B))
        continue;
    vector<string> beta(rhs.begin() + it.d + 1, rhs.end());
    beta.push_back(it.la);
    for (auto &b : firstSeq(beta))
    {
        if (b == "eps")
            continue;
        for (int i = 0; i < (int)G.size(); i++)
        {
            if (G[i].first != B)
                continue;
            Item ni{i, 0, b};
            if (!s.count(ni))
            {
                s.insert(ni);
                wl.push_back(ni);
            }
        }
```

```
        }
    }
    return s;
}

ISet gotoSt(const ISet &s, const string &sym)
{
    ISet kernel;
    for (auto &it : s)
    {
        auto &rhs = G[it.p].second;
        if (it.d < (int)rhs.size() && rhs[it.d] == sym)
            kernel.insert({it.p, it.d + 1, it.la});
    }
    return kernel.empty() ? kernel : closure1(kernel);
}

vector<ISet> STATES;
map<pair<int, string>, int> TRANS;

void buildCollection()
{
    STATES.push_back(closure1({{0, 0, "$"}}));
    bool changed = true;
    while (changed)
    {
        changed = false;
        int n = (int)STATES.size();
```

```
for (int i = 0; i < n; i++)
{
    set<string> syms;
    for (auto &it : STATES[i])
    {
        auto &rhs = G[it.p].second;
        if (it.d < (int)rhs.size())
            syms.insert(rhs[it.d]);
    }
    for (auto &sym : syms)
    {
        ISet g = gotoSt(STATES[i], sym);
        if (g.empty())
            continue;
        int idx = -1;
        for (int j = 0; j < (int)STATES.size(); j++)
            if (STATES[j] == g)
            {
                idx = j;
                break;
            }
        if (idx == -1)
        {
            idx = (int)STATES.size();
            STATES.push_back(g);
            changed = true;
        }
        TRANS[{i, sym}] = idx;
```

```
        }
      }
    }
}

map<int, map<string, string>> ACT;
map<int, map<string, int>> GOT;
vector<string> conflicts;

void buildTables()
{
    for (int i = 0; i < (int)STATES.size(); i++)
    {
        for (auto &it : STATES[i])
        {
            auto &[lhs, rhs] = G[it.p];
            if (it.d < (int)rhs.size())
            {
                const string &sym = rhs[it.d];
                if (TM.count(sym) && sym != "$")
                {
                    if (TRANS.count({i, sym}))
                    {
                        string act = "s" + to_string(TRANS[{i, sym}]);
                        if (ACT[i].count(sym) && ACT[i][sym] != act)
                            conflicts.push_back("State " + to_string(i) + " on '" + sym +
                                                "': " + ACT[i][sym] + " vs " + act);
                        ACT[i][sym] = act;
```

```
            }
          }
          else if (isNT(sym))
          {
            if (TRANS.count({i, sym}))
              GOT[i][sym] = TRANS[{i, sym}];
          }
        }
        else
        {
          if (lhs == AUG && it.la == "$")
          {
            ACT[i]["$"] = "acc";
          }
          else
          {
            string act = "r" + to_string(it.p);
            if (ACT[i].count(it.la) && ACT[i][it.la] != act)
              conflicts.push_back("State " + to_string(i) + " on '" + it.la +
                                  "': " + ACT[i][it.la] + " vs " + act);
            ACT[i][it.la] = act;
          }
        }
      }
    }
}

vector<int> colWidths(const vector<string> &hdr, const vector<vector<string>> &rows)
```

```
{
   vector<int> w(hdr.size(), 0);
   for (int i = 0; i < (int)hdr.size(); i++)
      w[i] = (int)hdr[i].size();
   for (auto &r : rows)
      for (int i = 0; i < (int)r.size() && i < (int)w.size(); i++)
         w[i] = max(w[i], (int)r[i].size());
   return w;
}

void hline(const vector<int> &w, char fc = '-')
{
   cout << "+";
   for (auto x : w)
      cout << string(x + 2, fc) << "+";
   cout << "\n";
}

void printRow(const vector<int> &w, const vector<string> &cells)
{
   cout << "|";
   for (int i = 0; i < (int)cells.size(); i++)
   {
      string c = i < (int)cells.size() ? cells[i] : "";
      cout << " " << left << setw(w[i]) << c << " |";
   }
   cout << "\n";
}
```

```
void printTable(const vector<string> &hdr, const vector<vector<string>> &rows)
{
   auto w = colWidths(hdr, rows);
   hline(w, '=');
   printRow(w, hdr);
   hline(w, '=');
   for (auto &r : rows)
   {
      printRow(w, r);
      hline(w, '-');
   }
}

void sectionBanner(const string &title)
{
   int n = (int)title.size() + 4;
   cout << "\n"
       << string(n, '=') << "\n";
   cout << "| " << title << " |\n";
   cout << string(n, '=') << "\n\n";
}

void printGrammar()
{
   sectionBanner("GRAMMAR  (Augmented)");
   for (int i = 0; i < (int)G.size(); i++)
   {
```

```
        cout << "  (" << i << ")  " << G[i].first << "  ->  ";
        if (G[i].second.empty())
            cout << "eps";
        else
            for (auto &s : G[i].second)
                cout << s << " ";
        cout << "\n";
    }
}

void printFirstSets()
{
    sectionBanner("FIRST SETS");
    vector<string> ntVec;
    for (auto &n : NT)
        if (n != AUG)
            ntVec.push_back(n);
    sort(ntVec.begin(), ntVec.end());
    for (auto &n : ntVec)
    {
        cout << "  FIRST( " << n << " )  =  { ";
        bool first = true;
        for (auto &x : FIRST[n])
        {
            if (!first)
                cout << ", ";
            cout << x;
            first = false;
```

```
      }
      cout << " }\n";
    }
}

void printStates()
{
    sectionBanner("CANONICAL  LR(1)  ITEM SETS");
    for (int i = 0; i < (int)STATES.size(); i++)
    {
      string hdr = "  State I" + to_string(i) + " ";
      cout << "\n"
          << hdr << string(max(0, 44 - (int)hdr.size()), '-') << "\n";
      for (auto &it : STATES[i])
        cout << "    " << itemStr(it) << "\n";
      bool any = false;
      for (auto &[key, val] : TRANS)
        if (key.first == i)
        {
          any = true;
          break;
        }
      if (any)
      {
        cout << "    " << string(36, '-') << "\n";
        for (auto &[key, val] : TRANS)
          if (key.first == i)
            cout << "    goto( " << key.second << " )  =>  I" << val << "\n";
```

```
        }
    }
}

void printParsingTable()
{
    sectionBanner("CLR(1)  PARSING  TABLE");

    vector<string> termCols, ntCols;
    for (auto &t : TM)
        if (t != "$")
            termCols.push_back(t);
    sort(termCols.begin(), termCols.end());
    termCols.push_back("$");

    for (auto &n : NT)
        if (n != AUG)
            ntCols.push_back(n);
    sort(ntCols.begin(), ntCols.end());

    vector<string> hdr = {"State"};
    for (auto &t : termCols)
        hdr.push_back(t);
    for (auto &n : ntCols)
        hdr.push_back(n);

    vector<vector<string>> rows;
    for (int i = 0; i < (int)STATES.size(); i++)
```

```
{
   vector<string> row = {to_string(i)};
   for (auto &t : termCols)
      row.push_back(ACT[i].count(t) ? ACT[i][t] : "-");
   for (auto &n : ntCols)
      row.push_back(GOT[i].count(n) ? to_string(GOT[i][n]) : "-");
   rows.push_back(row);
}

auto w = colWidths(hdr, rows);

int stateW = w[0];
int actionW = 0;
for (int i = 0; i < (int)termCols.size(); i++)
   actionW += w[1 + i] + 3;
actionW -= 1;
int gotoW = 0;
for (int i = 0; i < (int)ntCols.size(); i++)
   gotoW += w[1 + (int)termCols.size() + i] + 3;
gotoW -= 1;

cout << "+" << string(stateW + 2, '-')
    << "+" << string(actionW + 2, '-')
    << "+" << string(gotoW + 2, '-') << "+\n";
cout << "| " << left << setw(stateW) << "State"
    << " | " << left << setw(actionW) << "ACTION"
    << " | " << left << setw(gotoW) << "GOTO" << " |\n";
```

```
hline(w, '=');
printRow(w, hdr);
hline(w, '=');
for (auto &r : rows)
{
   printRow(w, r);
   hline(w, '-');
}

cout << "\n  Legend:\n";
cout << "    sN  = shift to state N\n";
cout << "    rN  = reduce by production (N)\n";
cout << "    acc = accept\n";
cout << "    -   = error / no entry\n";
cout << "\n  Productions used in reduce actions:\n";
for (int i = 0; i < (int)G.size(); i++)
{
   cout << "    (" << i << ")  " << G[i].first << "  ->  ";
   if (G[i].second.empty())
      cout << "eps";
   else
      for (auto &s : G[i].second)
         cout << s << " ";
   cout << "\n";
}
if (!conflicts.empty())
{
   cout << "\n  [!] CONFLICTS (grammar may not be CLR(1)):\n";
```

```
    for (auto &c : conflicts)
        cout << "     " << c << "\n";
  }
}

void parseInput()
{
  sectionBanner("PARSING  INPUT:  \"" + INPUT + "\"");

  string inp = INPUT + "$";
  int ip = 0;

  stack<int> stateStk;
  stack<string> symStk;
  stateStk.push(0);

  auto stackStr = [&]() -> string
  {
    vector<int> v;
    stack<int> tmp = stateStk;
    while (!tmp.empty())
    {
      v.push_back(tmp.top());
      tmp.pop();
    }
    reverse(v.begin(), v.end());
    string s;
    for (auto x : v)
```

```
        s += to_string(x) + " ";
    return s;
};

auto symStr = [&]() -> string
{
    vector<string> v;
    stack<string> tmp = symStk;
    while (!tmp.empty())
    {
        v.push_back(tmp.top());
        tmp.pop();
    }
    reverse(v.begin(), v.end());
    string s = "$";
    for (auto &x : v)
        s += x;
    return s;
};

vector<string> hdr = {"Step", "State Stack", "Symbol Stack", "Input", "Action"};
vector<vector<string>> rows;

int step = 0;
bool accepted = false;
bool errored = false;

while (true)
```

```
{
    step++;
    int state = stateStk.top();
    string tok(1, inp[ip]);

    if (!ACT[state].count(tok))
    {
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), "ERROR"});
        errored = true;
        break;
    }

    string act = ACT[state][tok];

    if (act == "acc")
    {
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), "ACCEPT"});
        accepted = true;
        break;
    }

    if (act[0] == 's')
    {
        int ns = stoi(act.substr(1));
        string desc = "Shift '" + tok + "'  -> push s" + to_string(ns);
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), desc});
        stateStk.push(ns);
        symStk.push(tok);
```

```
        ip++;
      }
      else
      {
        int pid = stoi(act.substr(1));
        auto &[lhs, rhs] = G[pid];
        string prodStr = lhs + " ->";
        for (auto &s : rhs)
          prodStr += " " + s;
        string desc = "Reduce by (" + to_string(pid) + ") " + prodStr;
        rows.push_back({to_string(step), stackStr(), symStr(), inp.substr(ip), desc});
        for (int i = 0; i < (int)rhs.size(); i++)
        {
          stateStk.pop();
          symStk.pop();
        }
        int top = stateStk.top();
        if (!GOT[top].count(lhs))
        {
          rows.push_back({to_string(step + 1), stackStr(), symStr(), inp.substr(ip), "GOTO
ERROR"});
          errored = true;
          break;
        }
        stateStk.push(GOT[top][lhs]);
        symStk.push(lhs);
      }
    }
```

```
    printTable(hdr, rows);

    cout << "\n";
    if (accepted)
        cout << "  >> RESULT: \"" << INPUT << "\" is ACCEPTED by the grammar.\n\n";
    else
        cout << "  >> RESULT: \"" << INPUT << "\" is REJECTED (syntax error at step "
            << step << ").\n\n";
}

int main()
{
    deriveSymbols();
    computeFirst();
    buildCollection();
    buildTables();

    printGrammar();
    printFirstSets();
    printStates();
    printParsingTable();
    parseInput();

    return 0;
}
```

Input and Output:

Input Grammar: (NT = {c , d}, T = {S, C }, P={ S -> CC, C -> cC, C -> d}, S = {S})

Input String: ccdd

```
PS C:\Users\vishw> cd "c:\Users\vishw\Coding\Compiler-Lab\Week-9\" ; if ($?) { g++ clr_parser_main.cpp -o clr_parser_main } ; if ($?) { .\clr_parser_main }

========================
| GRAMMAR  (Augmented) |
========================

  (0)  S'  ->  S
  (1)  S  ->  C C
  (2)  C  ->  c C
  (3)  C  ->  d

==============
| FIRST SETS |
==============

  FIRST( C )  =  { c, d }
  FIRST( S )  =  { c, d }

==============================
| CANONICAL  LR(1)  ITEM SETS |
==============================


  State I0 --------------------------------
    S' -> . S  [$]
    S -> . C C  [$]
    C -> . c C  [c]
    C -> . c C  [d]
    C -> . d  [c]
    C -> . d  [d]
    ------------------------------------
    goto( C )  =>  I1
    goto( S )  =>  I2
    goto( c )  =>  I3
    goto( d )  =>  I4
```

```
State I1 ---------------------------------
  S -> C . C  [$]
  C -> . c C  [$]
  C -> . d  [$]
  -------------------------------------
  goto( C )  =>  I5
  goto( c )  =>  I6
  goto( d )  =>  I7

State I2 ---------------------------------
  S' -> S .  [$]

State I3 ---------------------------------
  C -> . c C  [c]
  C -> . c C  [d]
  C -> c . C  [c]
  C -> c . C  [d]
  C -> . d  [c]
  C -> . d  [d]
  -------------------------------------
  goto( C )  =>  I8
  goto( c )  =>  I3
  goto( d )  =>  I4

State I4 ---------------------------------
  C -> d .  [c]
  C -> d .  [d]

State I5 ---------------------------------
  S -> C C .  [$]

State I6 ---------------------------------
  C -> . c C  [$]
  C -> c . C  [$]
  C -> . d  [$]
  -------------------------------------
  goto( C )  =>  I9
  goto( c )  =>  I6
  goto( d )  =>  I7
```

```
  State I7 ---------------------------------
    C -> d .  [$]

  State I8 ---------------------------------
    C -> c C .  [c]
    C -> c C .  [d]

  State I9 ---------------------------------
    C -> c C .  [$]

==========================
| CLR(1)  PARSING  TABLE |
==========================

+-------+-----------------+---------+
| State | ACTION          | GOTO    |
+=======+====+====+=====+===+===+
| State | c  | d  | $   | C | S |
+=======+====+====+=====+===+===+
| 0     | s3 | s4 | -   | 1 | 2 |
+-------+----+----+-----+---+---+
| 1     | s6 | s7 | -   | 5 | - |
+-------+----+----+-----+---+---+
| 2     | -  | -  | acc | - | - |
+-------+----+----+-----+---+---+
| 3     | s3 | s4 | -   | 8 | - |
+-------+----+----+-----+---+---+
| 4     | r3 | r3 | -   | - | - |
+-------+----+----+-----+---+---+
| 5     | -  | -  | r1  | - | - |
+-------+----+----+-----+---+---+
| 6     | s6 | s7 | -   | 9 | - |
+-------+----+----+-----+---+---+
| 7     | -  | -  | r3  | - | - |
+-------+----+----+-----+---+---+
| 8     | r2 | r2 | -   | - | - |
+-------+----+----+-----+---+---+
| 9     | -  | -  | r2  | - | - |
+-------+----+----+-----+---+---+
```

```
Legend:
  sN  = shift to state N
  rN  = reduce by production (N)
  acc = accept
  -   = error / no entry

Productions used in reduce actions:
  (0)  S'  ->  S
  (1)  S  ->  C C
  (2)  C  ->  c C
  (3)  C  ->  d
```

```
===========================
| PARSING  INPUT:  "ccdd" |
===========================

+======+=============+==============+=======+========================+
| Step | State Stack | Symbol Stack | Input | Action                 |
+======+=============+==============+=======+========================+
| 1    | 0           | $            | ccdd$ | Shift 'c'  -> push s3  |
+------+-------------+--------------+-------+------------------------+
| 2    | 0 3         | $c           | cdd$  | Shift 'c'  -> push s3  |
+------+-------------+--------------+-------+------------------------+
| 3    | 0 3 3       | $cc          | dd$   | Shift 'd'  -> push s4  |
+------+-------------+--------------+-------+------------------------+
| 4    | 0 3 3 4     | $ccd         | d$    | Reduce by (3) C -> d   |
+------+-------------+--------------+-------+------------------------+
| 5    | 0 3 3 8     | $ccC         | d$    | Reduce by (2) C -> c C |
+------+-------------+--------------+-------+------------------------+
| 6    | 0 3 8       | $cC          | d$    | Reduce by (2) C -> c C |
+------+-------------+--------------+-------+------------------------+
| 7    | 0 1         | $C           | d$    | Shift 'd'  -> push s7  |
+------+-------------+--------------+-------+------------------------+
| 8    | 0 1 7       | $Cd          | $     | Reduce by (3) C -> d   |
+------+-------------+--------------+-------+------------------------+
| 9    | 0 1 5       | $CC          | $     | Reduce by (1) S -> C C |
+------+-------------+--------------+-------+------------------------+
| 10   | 0 2         | $S           | $     | ACCEPT                 |
+------+-------------+--------------+-------+------------------------+

  >> RESULT: "ccdd" is ACCEPTED by the grammar.
```

Result:

The CLR parser has been successfully written in C++ and executes as expected.